SUMMARY
Advance Machine learning
Cats Vs Dogs- CNN

This report summarizes the findings from training a convolutional neural network (CNN) from scratch on the Cats & Dogs dataset and then leveraging a pretrained VGG16 model for feature extraction and fine-tuning. The study aimed to analyze the relationship between training sample size, model choice, and performance, highlighting the importance of balancing data size and model complexity to optimize prediction results.

The model development progressed through three main stages, each with different training sample sizes, culminating in comparison with a pretrained VGG16 model. For each stage, dropout layers and data augmentation were applied to reduce overfitting, and model checkpoints preserved the best-performing model based on validation loss. In the final stage, the VGG16 pretrained model, initialized with ImageNet weights, was used for feature extraction. Specific layers were unfrozen for fine-tuning, enhancing performance through transfer learning.

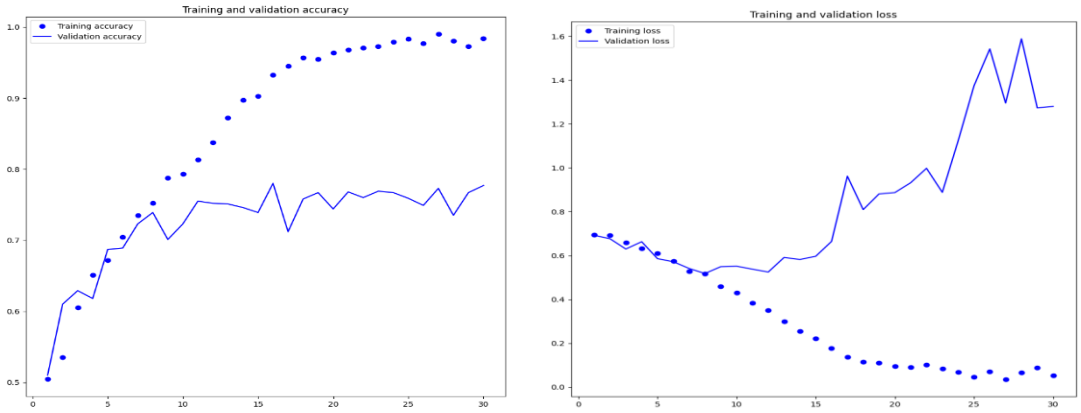
Performance Summary Table

Model Type	Training Sample Size	Training Accuracy	Training Accuracy	Test Accuracy	Overfitting Observed
Training from Scratch	1000	99%	77.7%	72%	High
Training from Scratch	1500	75.9%	73.5%	73.8%	Moderate
Training from Scratch	2000	75.1%	76.6%	72.4%	Reduced
VGG16 (Feature Extraction)	2000	99.9%	96.2%	97.5%	Minimal
VGG16 (Fine-Tuning)	2000	98.2%	97.3%	96.9%	Minimal

Findings and Plot Analysis

In the initial stage, I trained the model with 1000 training samples, 500 for validation, and 500 for testing. To tackle overfitting, I introduced a 50% dropout rate and set up a model checkpoint to save the best version based on validation loss. While the training accuracy soared to nearly 99%, the validation accuracy stagnated at around 77.7%. This pointed to overfitting, which likely resulted from the limited amount of training data.

This overfitting was evident in the plots, where training accuracy steadily increased while validation accuracy stopped improving after a certain point. The validation loss initially decreased but began to rise after epoch 15, reinforcing that the model struggled to generalize effectively to new data.

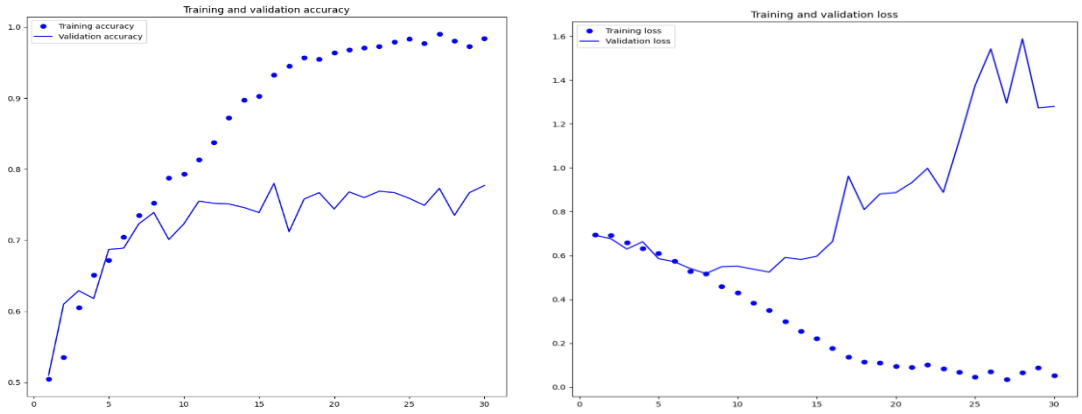


While the model achieved high training accuracy, its performance on validation data showed limited generalization, highlighting the challenge of overfitting in training from scratch on small datasets. Techniques like dropout helped mitigate overfitting, but using a pretrained model may improve generalization further in future trials.

Next, I increased the training dataset to 1500 samples while keeping the validation and test samples the same. I also added data augmentation, including horizontal flipping, rotation, and zooming, to further combat overfitting. With the larger dataset, the training accuracy improved to 75.9%, and validation accuracy reached around 73.5%, showing better generalization compared to the smaller dataset.

I went a step further by increasing the training sample size to 2000. With this larger dataset, the model achieved a training accuracy of 75.1% and validation accuracy of 76.6%. The test accuracy was 72.4%, indicating that the model was generalizing well. The plots showed a steady increase in both training and validation accuracy, and the validation loss decreased more smoothly compared to earlier steps. This suggests that 2000 samples might be an optimal training size for this task.

For the final experiment, I used the VGG16 model with pretrained ImageNet weights for feature extraction. The dataset passed through the convolutional layers of VGG16, and I used the extracted features to train a classifier. To further improve generalization, I applied data augmentation again.



The performance of the model was measured using accuracy and loss over multiple epochs. Training accuracy steadily increased, but validation accuracy fluctuated slightly. The validation accuracy remained high, suggesting strong generalization. By the end, the test accuracy was 0.975, indicating that the pretrained VGG16 model performed exceptionally well.

The loss plot showed a steady decrease, with occasional spikes. Fine-tuning the model by unfreezing a few layers helped improve accuracy even more, reaching a final test accuracy of about 0.9691. This suggests that fine-tuning the pretrained network can lead to better results than using it solely for feature extraction. However, unfreezing too many layers can risk overfitting, so this step requires caution.

Overall, the model's performance clearly improved as the training data size increased. This experiment demonstrated that using larger datasets, combined with techniques like dropout and data augmentation, helps reduce overfitting and improve generalization. This iterative approach showed me that optimizing the training sample size and choosing the right optimizer are key to getting the best results, whether training from scratch or using a pretrained model.